

05-Use-Cases-and-User-Stories

- Use Cases & User Stories — Leish! v2
- 1. Use-Case Summary (Umbrella)
 - 2. Detailed Use Cases
 - 3. User Stories (INVEST) — Backlog Format
 - 4. Non-Functional Stories (Supplementary)
 - 5. Trace to FRS

Use Cases & User Stories — Leish! v2

FIELD	VALUE
Document ID	LEISH-UCS-v2.0
Version	2.0.0
Date	2026-08-29
Predecessor	BRS (01), SRS (02), FRS (03)
Notation	UC-XX use cases (Cockburn), US-XX stories (INVEST)

1. Use-Case Summary (Umbrella)



ID	USE CASE	PRIMARY ACTOR	PRIORITY
UC-01	Register & Verify Email	Guest	Must
UC-02	Login & Manage Session	Guest/Customer/Artist/Admin	Must
UC-03	Reset Forgotten Password	Guest	Must
UC-04	Discover Artists / Studios	Guest/Customer	Must
UC-05	View Artist/Studio Detail & Reviews	Guest	Must
UC-06	Submit Booking Request	Customer	Must
UC-07	Accept / Reject Booking	Artist	Must
UC-08	Build & Send Quotation	Artist	Must
UC-09	Pay Deposit (Booking Fee) & Confirm	Customer	Must
UC-10	Pay Balance (T-3d) & Trigger Payout	Customer	Must
UC-11	Cancel Booking / Request Refund	Customer / Artist	Must
UC-12	Complete Booking & Review	Customer / Artist	Must
UC-13	Chat Per-Booking (Live SSE)	Customer / Artist	Must
UC-14	Claim Artist/Studio Profile	Artist / Studio	Must
UC-15	Manage Own Artist Profile (Self-Service)	Artist / Studio	Should
UC-16	Admin Dashboard & User Management	Admin	Must
UC-17	Admin Catalog & Booking Overrides	Admin	Must
UC-18	Admin Payout Settlement	Admin	Must
UC-19	Export Data & Delete Account (PDPA)	Customer	Must
UC-20	Manage Email Preferences / View Outbox	Customer / Admin	Should

2. Detailed Use Cases

UC-01 — REGISTER & VERIFY EMAIL

Primary actor: Guest **Preconditions:** No active session; `LEISH_DB_PATH` reachable or PG available. **Success guarantee:** Account created with role, `email_verified=0`, verification token stored hashed.

STEP	ACTOR	ACTION
1	Guest	Opens <code>/register</code> , chooses role (Client/Artist/Studio toggles), enters name/email/password, checks PDPA consent, optionally solves Turnstile.
2	Guest	Submits; client validates inline via <code>registerSchema</code> .
3	System	Validates zod, lowercases email, checks uniqueness, hashes with scrypt+pepper, inserts <code>users</code> + <code>email_verifications</code> (hashed token), sets <code>leish_session</code> JWT cookie, enqueues verification email (or writes <code>email_outbox</code>).
4	System	Redirects to <code>/dashboard</code> ; banner "Verify your email" with resend button.
5	Guest	Opens email, clicks <code>/verify-email?token=...</code>
6	System	Validates single-use, non-expired; sets <code>email_verified=1</code> , marks <code>used_at</code> , redirects with success.

Alternate flows: - 1a Duplicate email → 409 generic error (no enumeration). - 3a Email provider down → dev outbox + warn log; dashboard still shows `devVerifyUrl`. - 5a Expired token → friendly "Link expired, request a new one."

Acceptance: See FRS FR-AUTH-01/05.

UC-02 — LOGIN & SESSION

Primary actor: Any authed role **Includes:** login, session refresh via middleware `proxy.ts`, logout/revocation.

STEP	ACTION
1	User opens <code>/login</code> (or redirected from gated route with <code>?redirect=/dashboard</code>).
2	Enters email/password (+ Turnstile when configured).
3	System verifies scrypt+pepper, creates <code>sessions(jti,...)</code> , signs JWT, sets cookie. <code>agnost.begin(login)</code> ends success=true.
4	Redirect to <code>redirect</code> or <code>/dashboard</code> with role label.
5	Later: visits any route — <code>verifySessionToken</code> checks sig+expiry+revoked; protected routes (<code>/dashboard</code> , <code>/onboarding</code> , <code>/admin/*</code>) enforce via middleware layout.
6	Click Log out → <code>POST /api/auth/logout</code> → <code>UPDATE sessions SET revoked=1</code> + clear cookie → redirect <code>/</code> .

Alternate: Wrong creds → 401 generic; rate-limited → 429 with Retry-After; revoked JTI → redirect to login.

UC-03 — RESET FORGOTTEN PASSWORD

STEP	ACTION
1	Guest on <code>/forgot-password</code> enters email (+ Turnstile).
2	System: rate-limit by IP, lookup user (always reply success-ish), if exists generate <code>password_resets</code> hashed token 1h, send email.
3	Guest clicks <code>/reset-password?token=...</code> , enters new password (8–128).
4	System verifies single-use + expiry, script new hash, marks <code>used_at</code> , invalidates other reset tokens. Can now log in.

Alternate: Expired token → error; reused token → `used_at` blocks.

UC-04 — DISCOVER ARTISTS / STUDIOS

Actors: Guest / Customer **Entry:** `/`, `/artists`, `/studios`

STEP	ACTION
1	Page server-fetches <code>ListAllArtists()</code> (DB-backed, <code>force-dynamic</code>) and renders <code>ArtistsBrowser</code> .
2	User interacts: type query (name/tagline/specialties), pick <code>state</code> (cascades <code>area</code> list from <code>AREAS_BY_STATE</code>), pick <code>bridal</code> / <code>nonBridal</code> tag, set <code>budget</code> , pick <code>date</code> . <code>trackSearch/trackArtistView</code> fire via <code>agnost-client</code> when configured.
3	System: <code>artistsQuerySchema</code> validates → <code>ListArtists(filters)</code> SQL pre-filters <code>state/area/budget</code> then <code>filterArtists</code> for remaining predicates — returns ranked by <code>rating DESC</code> .
4	User sees filtered cards (<code>ArtistCard</code>) with image, rating, priceFrom, area; clicks → <code>/artists/[slug]</code> .
5	Empty result → friendly “No artists found, try adjusting filters.”

UC-05 — VIEW ARTIST/STUDIO DETAIL

STEP	ACTION
1	User opens <code>/artists/aisha-azman</code> (slug=legacy id). Server resolves <code>resolveArtist(slug)</code> → <code>rowToArtist</code> + <code>ListEntityReviews</code> .
2	Page shows bio, specialties tags, services table (name/price/duration), portfolio carousel, availability slots, reviews (<code>RatingStars</code>), "Book Now" CTA.
3	"Book Now" requires auth → if guest → redirect <code>/login?redirect=/artists/[slug]/book</code> .

Alternate: Unknown slug → `notFound()` true 404 (no shell 200).

UC-06 — SUBMIT BOOKING REQUEST

Primary actor: Customer (verified for next step, but request allowed even if unverified per FRS FR-BOOK-01)

Entry: `/artists/[slug]/book` or booking modal on detail page.

STEP	ACTION
1	Customer picks <code>service</code> (from artist.services), <code>date</code> (≥today, datepicker), <code>time</code> (slot), enters <code>eventType</code> (select—bridal/nonBridal taxonomy), <code>venue</code> , <code>guestCount</code> , <code>notes</code> .
2	Validates <code>bookingSchema</code> ; client price hidden (server resolves).
3	Submits <code>POST /api/bookings</code> ; system pre-checks slot, inserts <code>bookings(status=requested)</code> , emails artist, Agnost <code>booking-create</code> .
4	Redirect to <code>/dashboard</code> where booking appears <code>requested</code> amber badge.

Alternates: - 3a Slot taken (unique index race) → 409 "just been taken, pick another time". - 3b Past date → 400 "Date cannot be in the past". - 3c Invalid service → 400 "Service not available for this artist".

UC-07 — ACCEPT / REJECT BOOKING

Primary actor: Artist (claimed)

STEP	ACTION
1	Artist sees <code>requested</code> bookings in dashboard filtered to claimedIds.
2	Clicks Accept → <code>PATCH /api/bookings/[id] {action:"accept"} →</code> <code>applyBookingTransition(requested,accept,{role:artist})</code> checks <code>isArtist</code> true and <code>current≡requested</code> → <code>accepted</code> ; or Reject → <code>cancelled</code> (terminal).
3	System updates row, optionally emails customer, shows new badge (sky <code>accepted</code> or red <code>cancelled</code>).

Alternates: Customer trying accept → “Only artists...”; accept on non-requested → error.

UC-08 — BUILD & SEND QUOTATION

Primary actor: Artist (claimed), booking must be `accepted` and not already pending quote (re-quote supersedes).

STEP	ACTION
1	Artist on booking card <code>accepted</code> without quotation sees “Build & send quotation”; clicks opens form.
2	Enters <code>baseFee</code> (required), optional <code>travelFee/earlyCallFee/accommodationFee</code> , adds ≤10 extras (label+amount), optional <code>artistNote</code> . Preview total auto-summed.
3	POST <code>/api/bookings/[id]/quotation</code> → system computes <code>quotationTotal()</code> , supersedes prior <code>pending</code> , inserts with <code>expires_at=now+24h</code> .
4	Customer dashboard now shows quotation breakdown, total, expiry countdown; server computed <code>balanceAmount = total - fee</code> and <code>balanceDueDate = eventDate - 3d</code> .

Alternate: Re-quote while `pending` → new row replaces, old becomes `superseded` ; quote after expiry (status `expired`) → new `pending` again allowed.

UC-09 — PAY DEPOSIT (BOOKING FEE) & CONFIRM

Primary actor: Customer

STEP	ACTION
1	Customer sees <code>accepted</code> + <code>quotation pending</code> + <code>Deposit status required</code> + "Pay RM 200 booking fee →" (+ Turnstile).
2	Click <code>Pay</code> → <code>POST /api/bookings/[id]/pay-fee</code> → <code>createBookingPayment('deposit', getBookingFeeSen())</code> ; system creates <code>payments(type=deposit, status=required)</code> ; if Billplz, returns <code>provider_url</code> Billplz hosted page.
3a	Dev: auto <code>markBillPaid</code> + <code>handlePaymentPaid</code> → <code>UPDATE bookings SET status='confirmed'</code> immediately; dashboard reloads → emerald <code>confirmed</code> + "✓ Booking fee paid".
3b	Billplz: open <code>provider_url</code> in new tab; after Billplz checkout, webhook <code>POST /api/payments/webhook</code> verified → same state change. Redirect landing <code>/dashboard</code> or <code>/booking/success</code> reads real status.
4	Fee is disclosed non-refundable throughout.

Alternate: Quotation expired → `totalPrice` null; pay-fee blocked until re-quoted.

UC-10 — PAY BALANCE (T-3D) & TRIGGER PAYOUT

STEP	ACTION
1	On <code>confirmed</code> booking, dashboard shows "Balance RM X due by <date (eventDate-3d)>" and "Pay balance" (when <code>balancePayment.status !== paid</code>).
2	Click <code>Pay balance (RM ...)</code> → <code>POST /api/bookings/[id]/pay-balance</code> → <code>amount = quotation.total - deposit</code> ; creates <code>payments(type=balance, status=required)</code> ; same Billplz/dev branching.
3	Dev auto-settle → <code>handlePaymentPaid(balance)</code> does <code>UPDATE quotations SET status='paid'</code> + <code>createPayoutForBooking</code> ; payouts row appears <code>pending</code> in admin.
4	Billplz webhook same logic; then payout <code>settleable_at = eventDate + 24h</code> .

Alternate: Zero balance (tiny quotation) → no payment needed / informative state.

UC-11 — CANCEL BOOKING / REQUEST REFUND

ACTOR	ACTION
Customer	On <code>requested accepted confirmed</code> sees Cancel → <code>PATCH {action:"cancel"}</code> → if <code>completed/cancelled</code> already → "no longer be cancelled"; else → <code>cancelled</code> .
Artist	Same; either side may cancel non-terminal; <code>isOwner</code> or <code>isArtist</code> suffices.
Refund	After <code>cancelled</code> that was <code>confirmed</code> and had <code>balance paid</code> : Customer sees Request refund → <code>POST /api/bookings/[id]/refund</code> → system refunds <code>balance</code> only; deposit stays. <code>dev</code> marks <code>refunded</code> directly; <code>billplz</code> calls refund API.

UC-12 — COMPLETE BOOKING & REVIEW

STEP	ACTION
1	Artist on <code>confirmed</code> clicks Complete → <code>PATCH {action:"complete"}</code> → <code>applyBookingTransition(confirmed,complete)</code> → <code>completed</code> (terminal).
2	Customer sees booking <code>completed</code> ; eligible for review: <code>findReviewableBooking</code> returns it (most recent completed without review).
3	Customer posts <code>POST /api/artists/[slug]/reviews {rating 1-5, text, event}</code> → <code>addEntityReview</code> guards duplicate, <code>INSERT reviews</code> , <code>blendAggregate</code> atomically updates artist <code>rating/review_count</code> .
4	New review appears on artist detail via <code>listEntityReviews</code> ordered desc.

Alternate: Studio bookings currently not reviewable (returns null). Second review for same booking → `ALREADY_REVIEWED` 409.

UC-13 — CHAT PER-BOOKING (LIVE SSE)

STEP	ACTION
1	On booking card, either party clicks "Chat" → collapsible <code>ChatThread</code> opens.
2	Component mounts SSE: <code>new EventSource(/api/bookings/[id]/messages/stream)</code> ; server replays history ordered ASC, then streams live via <code>chat-bus</code> pub/sub.
3	User types draft, POST <code>/api/bookings/[id]/messages {body}</code> → persisted + published → other participant receives event without refresh.
4	Authz: non-participant GET/POST returns 403.

UC-14 — CLAIM ARTIST/STUDIO PROFILE

STEP	ACTION
1	Artist/Studio registers → sees dashboard block "Your artist profile" + <code><select></code> of all artists from <code>/api/catalog/artists</code> + Claim profile button.
2	POST <code>/api/artist-profiles {artistId}</code> → checks role, uniqueness, inserts <code>artist_profiles(user_id,artist_id)</code> .
3	Future <code>GET /api/bookings</code> for that user is scoped to claimed ids; bookings for unclaimed artists remain invisible.

Alternate: Same `artist_id` already claimed by another user → 409; same `user_id` tries second profile → PK violation 409.

UC-16 — ADMIN DASHBOARD & USER MANAGEMENT

Actor: Admin **Entry:** `/admin` (layout guard redirects non-admins).

STEP	ACTION
1	Admin logs in (<code>role=admin</code> — seeded via <code>ADMIN_EMAIL/ADMIN_PASSWORD npx tsx scripts/seed-admin.ts</code>).
2	<code>/admin</code> fetch <code>GET /api/admin</code> → stats (users by role, bookings by status, payments revenue, recent lists) render <code>StatCard</code> s + tables.
3	<code>/admin/users</code> → list, create, <code>PATCH /api/admin/users/[id]</code> role/status edit, <code>DELETE</code> — every mutation audited (<code>admin_audit_log</code>).
4	Non-admin direct API call blocked 403 even without UI.

UC-18 — ADMIN PAYOUT SETTLEMENT

STEP	ACTION
1	Admin opens <code>/admin/payouts</code> ; <code>GET /api/admin/payouts?status=pending</code> lists payouts joined with booking+artist name.
2	Click Settle or Fail → <code>PATCH /api/admin/payouts {payoutId, status, notes}</code> → <code>updatePayoutStatus</code> sets <code>settled_at</code> when settled, logs audit.
3	Payout starts <code>pending</code> with <code>settleable_at = eventDate+24h</code> dispute window; settling before window shows warning but is allowed.

UC-19 — EXPORT DATA & DELETE ACCOUNT (PDPA)

STEP	ACTION
1	Any authed user on <code>/dashboard</code> "Your data" section clicks Export my data → <code>GET /api/me/export</code> downloads <code>leish-data-...json</code> (bookings+messages+payments+profile+preferences).
2	Click Delete my account → confirm dialog → <code>DELETE /api/me?confirm=1</code> → cascade via FK deletes owned data; anchor audit rows kept with <code>admin_user_id=null</code> ; cookie cleared; redirect <code>/</code> .

3. User Stories (INVEST) — Backlog Format

Stories complement use cases for sprint planning. Acceptance is Given/When/Then.

EPIC 1: DISCOVERY

ID	STORY	POINTS	PRIORITY	ACCEPTANCE
US-01	As a Guest , I want to browse featured artists on the home page, so I can quickly see top MUAs.	3	Must	Given  loads, when page renders, then top 3 by rating shown with hero/category/how-it-works/CTA.
US-02	As a Customer , I want to filter artists by state/area/budget/event , so I can find local affordable options.	5	Must	G/W filter state Selangor Then <code>ListArtists(state=Selangor)</code> returns only Selangor; area dropdown cascades.
US-03	As a Customer , I want to search artists by name/tagline/specialty text, so I can find known MUAs.	2	Must	G/W query "Aisha" Then results include Aisha Azman.

EPIC 2: ACCOUNT

ID	STORY	POINTS	ACCEPTANCE
US-04	As a Guest , I want to register as customer/artist/studio with PDPA consent, so I can transact.	3	See UC-01; zod errors inline; scrypt hashed.
US-05	As a User , I want to verify my email via link and have a resend button, so I can trust my account.	3	See UC-01 steps 5–6; links single-use hashed.
US-06	As a User , I want to log in/out and stay signed in 7 days across tabs, so I'm not re-authing constantly.	2	JWT cookie httpOnly; revoked JTI rejected.
US-07	As a User , I want to reset forgotten password without leaking which emails exist.	5	Forgot always 200-ish + 1h single-use token.
US-08	As an Artist , I want to claim my profile to receive relevant bookings only.	3	See UC-14; scoped <code>GET /api/bookings</code> .

EPIC 3: BOOKING & MONEY

ID	STORY	POINTS	ACCEPTANCE
US-09	As a Customer , I want to request a booking with service/date/time/event details and have the slot locked , so I don't lose it.	5	Slot unique index + 409; price derived server-side.
US-10	As an Artist , I want to accept/reject requests for my claimed profile.	2	<code>applyBookingTransition</code> enforced.
US-11	As an Artist , I want to quote (base/travel/early/accommodation+extras) with 24h window and re-quote if needed.	8	<code>quotationTotal</code> sum; <code>expires_at=now+24h</code> ; superseded.
US-12	As a Customer , I want to pay deposit (RM50) via Billplz/dev and see booking confirmed immediately (webhook).	8	<code>pay-fee</code> creates Billplz bill; webhook verified timing-safe.
US-13	As a Customer , I want to pay balance (total-deposit) due 3 days before the event.	5	<code>balanceDueDate</code> = eventDate-3d; payout created on paid.
US-14	As a Customer , I want to cancel and get balance refunded (deposit non-refundable) if needed.	3	<code>refundBalancePayment</code> guards balance-only.
US-15	As an Artist , I want my payout computed correctly (waiver, commission) and settleable after 24h .	5	<code>computeCommission</code> + <code>net</code> = <code>total-commission-deposit</code> .

EPIC 4: SOCIAL & COMMS

ID	STORY	POINTS	ACCEPTANCE
US-16	As a Customer/Artist , I want per-booking chat that updates live without refresh.	5	SSE stream replay+live; 403 for outsiders.
US-17	As a Customer touching confirmed→completed , I want to review (once) and see rating update.	3	Gated on completed+unreviewed; atomic blend 2dp.
US-18	As a User , I want to control email prefs (toggle types) and not be spammed.	2	email_preferences toggles honored.
US-19	As a Customer , I want invoice HTML + PDF for completed/confirmed bookings.	3	GET /invoice + /invoice.pdf for owner/claimed only.

EPIC 5: ADMIN

ID	STORY	POINTS	ACCEPTANCE
US-20	As an Admin , I want a dashboard with user/booking/payment stats + recent activity.	3	GET /api/admin aggregates.
US-21	As an Admin , I want to CRUD users/artists/studios/bookings with all changes audited .	5	admin_audit_log on every mutation.
US-22	As an Admin , I want to settle/fail payouts after the dispute window.	3	PATCH /api/admin/payouts .
US-23	As an Admin , I want to tune platform settings (fee/commission/waiver) without redeploying.	2	PATCH /api/admin/settings + 30s cache.

EPIC 6: COMPLIANCE

ID	STORY	POINTS	ACCEPTANCE
US-24	As a Customer , I want to export my data as JSON and delete my account fully.	3	PDPA; cascade semantics.

4. Non-Functional Stories (Supplementary)

ID	STORY	ACCEPTANCE
NFR-US-01	As Engineering , I want CI to gate every PR on lint/typecheck/test/coverage≥80%/format, so debt doesn't ship.	<code>quality-gate.yml</code> required.
NFR-US-02	As Ops , I want logs structured then forwarded to webhook in batches, so I can pipe to ELK/Datadog.	<code>LOG_WEBHOOK_URL</code> 20/50ms batch.
NFR-US-03	As Security , I want security headers + CSP nonce + HMAC webhook, so attacks are rejected early.	<code>next.config.ts</code> headers + <code>verifyBillplzSignature</code> timing-safe.

5. Trace to FRS

UC	FRS IDS
UC-01	FR-AUTH-01, FR-AUTH-05
UC-02	FR-AUTH-02, FR-AUTH-03, FR-AUTH-04, FR-AUTH-07
UC-03	FR-AUTH-06
UC-04	FR-CAT-01, FR-CAT-03
UC-05	FR-CAT-02
UC-06	FR-BOOK-01, FR-DASH-01
UC-07	FR-BOOK-03
UC-08	FR-QUO-01, FR-QUO-02
UC-09	FR-PAY-02, FR-PAY-04
UC-10	FR-PAY-03, FR-PAY-04, FR-PAYO-02
UC-11	FR-BOOK-03, FR-PAY-05
UC-12	FR-BOOK-03, FR-REV-01..03
UC-13	FR-MSG-01, FR-MSG-02
UC-14	FR-CLAIM-01, FR-CLAIM-02
UC-16..18	FR-ADM-01..04, FR-PAYO-03
UC-19	FR-COMP-01, FR-COMP-02
UC-20	FR-MAIL-03, FR-ADM-02 (emails)

Full traceability lives in `docs/08-Traceability-Matrix.md`.

Next: `docs/06-Data-Model-and-ERD.md` specifies the storage for every entity referenced above.